DSA Interview Preparation Notes

Time Complexity & Space Complexity

Quick reference guide for placements and technical interviews

1. Fundamentals of Time Complexity

1.1 What is Time Complexity?

Time complexity tells you how long an algorithm takes to run as the input size grows. It is measured in three cases:

Case	Meaning	When to Use
Best Case	Minimum time taken	Input is already ideal (e.g., sorted list)
Average Case	Expected time on random input	Most practical scenarios
Worst Case	Maximum time taken	Always used in interviews and DSA problems

In interviews and DSA problems, we almost always use the Worst Case because it gives a guarantee on performance.

1.2 Big-O Notation

Big-O notation describes how the runtime grows with input size. We drop constants and smaller terms and keep only the dominant term.

Key rules:

- Drop constants: O(3N) becomes O(N)
- Drop smaller terms: $O(N + N^2)$ becomes $O(N^2)$
- Keep only the largest/dominant factor

1.3 Calculating Time Complexity - Basic Examples

Example 1: No input (constant)

```
a = 1
b = 1
print(a + b)
```

Each line takes O(1). Total = O(1) + O(1) + O(1) = 3 x O(1) = O(1)
Any fixed number of operations with no input is always O(1).

Example 2: Single loop

```
for i = 0 to N:
    print(i)
```

The loop runs N times. Each iteration does O(1) work. Total = N x O(1) = O(N).

Example 3: Two separate loops

```
for i = 0 to N:
    print(i)
for i = 0 to N:
    print(i)
```

First loop = O(N), second loop = O(N). Total = O(N) + O(N) = 2 x O(N) = O(N).
Sequential loops add, but constants are dropped.

Example 4: Nested loops (same variable)

```
for i = 0 to N:
    for j = 0 to N:
        for k = 0 to N:
            print(i)
```

Three nested loops each running N times = N x N x N = $O(N^3)$.

Example 5: Nested loops (different variables)

```
for i = 0 to N:
```

```
    for j = 0 to M:
        print(i, j)
```

When the loops use different variables, you cannot combine them. Result = O(N x M).

Example 6: Loops with constants - ignore them

```
for i = 1 to (N/10):
    for j = 1 to (2*M):
        for l = 1 to K:
            print(i, j, l)
```

Outer loop runs N/10 times → ignore constant → O(N)
Middle loop runs 2*M times → ignore constant → O(M)
Inner loop runs K times → O(K)
All nested → Total = O(N x M x K)

1.4 While Loop Complexity

For while loops, look at how the loop variable changes in each step:

Update Type	Example	Complexity
Add/Subtract a constant	i = i + 2	O(N)
Multiply/Divide by a constant	i = i * 2 or i = i / 2	O(log N)

While Loop Example

```
i = 1
while (i < N):
    i = i + 2
    j = 1
    while (j < M):
        j = j + 1
        print(i + j)
```

Outer loop: i goes from 1 to N in steps of 2 → runs N/2 times → O(N)
Inner loop: j goes from 1 to M in steps of 1 → runs M times → O(M)
Nested → Total = O(N/2 x M) = O(NM) after dropping constant.

2. Applying Time Complexity to DSA Problems

2.1 How to Decide if Your Solution is Fast Enough

In DSA problems, constraints tell you the maximum value of N. Use these steps to check if your algorithm will pass:

1. Find the worst case time complexity of your algorithm.
2. Substitute the maximum value of N from the problem constraints.
3. If the result is below 10^8 operations, your solution will likely pass. If it exceeds 10^8, you will get a TLE (Time Limit Exceeded) error.

Rule of thumb: A modern computer runs approximately 10^8 (100 million) operations per second.

2.2 Quick Reference - Max N for Each Complexity

Time Complexity	Max N (safe)	Notes
O(1)	Any	Constant time, always fine
O(log N)	10^{10-8} (very large)	Extremely efficient
O(N)	10^8	Single loop, linear search, etc.
O(N log N)	10^7	Merge sort, efficient sorting
$O(N^2)$	10^4	Nested loops, bubble/insertion sort
$O(N^3)$	500	Triple nested loops
$O(2^R)$	~20	Exponential, brute force only

2.3 Standard Algorithm Complexities

Loops

Algorithm	Complexity	Max N
Single loop till N	O(N)	10^8
Two separate loops till N	O(N)	10^8
Nested loops both till N	$O(N^2)$	10^4
Triple nested loops till N	$O(N^3)$	~500

Searching

Algorithm	Complexity	Max N
Linear Search	O(N)	10^8

Algorithm	Complexity	Max N
Binary Search	O(log N)	10^{10-8} (extremely large)

Sorting

Algorithm	Complexity	Max N
Bubble / Insertion Sort	$O(N^2)$	10^4
Merge Sort / Quick Sort / Heap Sort	O(N log N)	10^7

Array / List Operations

Operation	Complexity
Access by index	O(1)
Add element at the end	O(1)
Add element at the start	O(N)
Count occurrences in an array	O(N)

Data Structure Operations (Average Case)

Operation	Complexity
Array access	O(1)
Hash Table - Insert / Search / Delete	O(1)
Balanced BST - Insert / Search	O(log N)
Linked List - Access / Search	O(N)

3. Log and Square Root Time Complexity

3.1 O(log N) - Logarithmic Complexity

An algorithm is O(log N) when the input is divided (or multiplied) by a constant in each step. The most common case is dividing by 2.

Pattern: Loop variable is multiplied or divided each iteration.

```
i = 1
while (i < N):
    i = i * 2    // i doubles each time: 1, 2, 4, 8, 16 ...
```

If N = 1000, this loop runs about 10 times (since $2^{10} = 1024$). Total iterations = $\log_2(N) = O(\log N)$.

Common examples of O(log N):

- Binary Search
- Searching in a Balanced Binary Search Tree
- Dividing a problem in half each time (Divide and Conquer)

3.2 O(N log N) - Linearithmic Complexity

This appears when you do O(log N) work N times, or when you sort using an efficient algorithm.

Common examples:

- Merge Sort
- Quick Sort (average case)
- Heap Sort

3.3 O(√N) - Square Root Complexity

An algorithm is O(√N) when you only need to iterate up to the square root of N to find an answer. This is rare but appears in certain number theory problems.

```
i = 1
while (i * i <= N):
    // do something
    i = i + 1
```

The loop runs only √N times. Example use case: finding all divisors of a number N.

3.4 Common Complexity Patterns - Quick Summary

Pattern	Complexity
Iterating through input once	O(N)
Two separate loops over input	O(N)
Nested loops over same input	$O(N^2)$
Dividing input by half each step	O(log N)
Looping N times, each doing O(log N) work	O(N log N)
Loop until i*i <= N	O(√N)

4. Practice Problems

Try to find the time complexity of each problem before looking at the answer.

Problem 1

```
for i = 1 to N:
    for j = i to N:
        print(i, j)
```

Answer: $O(N^2)$. The inner loop runs fewer times as i increases, but the total number of iterations is approximately $N^2/2$. Drop the constant → $O(N^2)$.

Problem 2

```
i = N
while (i > 1):
    i = i / 2
```

Answer: O(log N). The variable i is halved each iteration: N, N/2, N/4 ... 1. That takes $\log_2(N)$ steps.

Problem 3

```
for i = 1 to N:
    j = 1
    while (j < N):
        j = j * 2
```

Answer: O(N log N). The outer loop runs N times. The inner while loop halves each time → O(log N). Total = N x O(log N) = O(N log N).

Problem 4

```
for i = 1 to (N/10):
    for j = 1 to (2*M):
        for l = 1 to K:
            print(i, j, l)
```

Answer: O(N x M x K). Ignore constants N/10 and 2*M. Three nested loops → multiply all three.

Problem 5

```
for i = 0 to N:
    for j = 0 to N:
```

```
        if (i == j):
            print(i)
```

Answer: $O(N^2)$. The if condition does not reduce the number of loop iterations. Both loops still run N times each.

5. Space Complexity

5.1 What is Space Complexity?

Space complexity is the amount of memory an algorithm needs to run, as a function of input size. It has two parts:

Type	Meaning	Example
Input Space	Memory used by the input data itself	The array passed to the function
Auxiliary Space	Extra memory used during computation	Temporary variables, new arrays, recursion stack

In most discussions, Space Complexity = Auxiliary Space. Input space is usually the same for all algorithms solving the same problem.

5.2 Space Complexity of Common Algorithms

Algorithm	Auxiliary Space	Input Space
Linear Search	O(1)	O(N)
Binary Search	O(1)	O(N)
Bubble Sort	O(1)	O(N)
Selection Sort	O(1)	O(N)
Insertion Sort	O(1)	O(N)
Merge Sort	O(N)	O(N)
Quick Sort (average)	O(log N)	O(N)
Quick Sort (worst)	O(N)	O(N)
Heap Sort	O(1)	O(N)
Radix Sort	O(N + K)*	O(N)
Counting Sort	O(N + M)*	O(N)
Bucket Sort	O(N)	O(N)

* K = range of input for Radix Sort. M = size of count array in Counting Sort.

5.3 Why O(1) Auxiliary Space?

An algorithm uses O(1) auxiliary space when it only uses a fixed number of extra variables regardless of input size. For example, bubble sort only needs one temporary variable to swap elements, no matter how large the array is.

5.4 Why Merge Sort uses O(N) Space

Merge sort creates a temporary array to merge the two halves. At any point, this temporary array can be as large as the entire input. Therefore its auxiliary space is O(N).

5.5 Recursion and Stack Space

Recursive algorithms use stack space equal to the depth of recursion:

- Quick Sort (average) recursion depth = O(log N) → Stack space = O(log N)
- Quick Sort (worst, unbalanced) recursion depth = O(N) → Stack space = O(N)
- Merge Sort recursion depth = O(log N) but uses extra array too → Total O(N)

5.6 Space vs Time Trade-off

Dynamic Programming (DP) is the classic example of trading space for speed:

- Without DP: Solve the same subproblem many times → slow but uses less memory
- With DP: Store intermediate results → faster but uses more memory

This is a common trade-off in algorithm design: you can often speed up an algorithm by using more memory, or save memory at the cost of doing more work.

5.7 Quick Interview Checklist for Space Complexity

Ask yourself	What to check
Are you creating a new array/list?	Size of that array → adds to space
Is the function recursive?	Recursion depth → adds to stack space
Are you using only a few variables?	O(1) auxiliary space
Are you using a hash map or set?	Space depends on number of unique elements stored

Quick Revision Cheat Sheet

Complexity	Name	Example Algorithm
$O(1)$	Constant	Array access, hash table lookup
$O(\log N)$	Logarithmic	Binary search, BST operations
$O(\sqrt{N})$	Square Root	Finding divisors of N
$O(N)$	Linear	Linear search, single loop
$O(N \log N)$	Linearithmic	Merge sort, Heap sort
$O(N^2)$	Quadratic	Bubble sort, nested loops
$O(N^3)$	Cubic	Triple nested loops
$O(2^R)$	Exponential	Brute force subsets

Remember: Always analyze Worst Case. If operations exceed 10^8, the solution will likely get TLE.